

ExcursionController.mjs

The excursion controller answers:

What SQS messages have I received?

A typical SQS event contains something like:

```
1 | event.Records
```

and each record has:

```
1 | record.body
```

The controller goes through the records.

Conceptually:

```
1 | for (  
2 |   const record of event.Records  
3 | ) {
```

Then:

```
1 | const message =  
2 |   JSON.parse(  
3 |     record.body  
4 |   );
```

Why JSON.parse again?

Because SQS has stored your message as text.

It then passes the message into:

```
1 | processExcursion(message)
```

which belongs to:

```
1 | excursionServices.mjs
```

So:

```
1 | excursionController  
2 |   ↓  
3 | processExcursion()  
4 |   ↓  
5 | excursionService
```

The old notes describe exactly this responsibility: read the SQS body, parse the JSON, then pass alert information to the service.

Controller result switch

Your excursion controller also uses a good `switch` case.

The service may return:

```
1 raised
2 duplicate
```

So:

```
1 switch (result.status) {
```

can handle:

```
1 case "raised":
```

meaning:

```
1 New alert successfully raised
```

and:

```
1 case "duplicate":
```

meaning:

```
1 This SQS message was already processed
```

That is a good use of a switch because `status` can be one of several known values.